

# Chapitre 21 - Parcours de graphes

## Objectifs :

- ▷ Implémenter en Python des algorithmes de parcours de graphes et de recherche de chemins dans un graphe.
- ▷ Utiliser ces fonctions Python pour résoudre un problème utilisant des graphes (navigation GPS, labyrinthes, mots voisins, ..., etc.)

## 1 Introduction

Un des premiers algorithmes que l'on doit savoir utiliser sur un graphe est celui de son parcours.

**Parcourir un graphe**, c'est visiter ses différents sommets, afin de pouvoir opérer une action tour à tour sur eux. Tous comme pour les arbres, il est possible de réaliser deux types de parcours d'un arbre :

- le **parcours en largeur** : *Breadth First Search*
- le **parcours en profondeur** : *Depth-First Search*

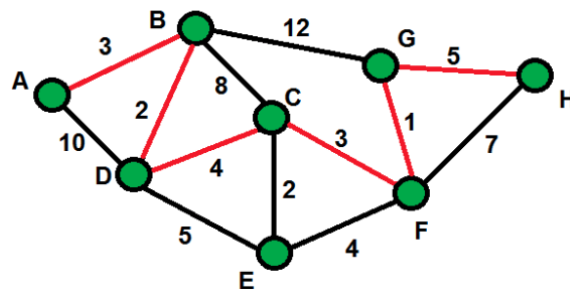
Cependant, contrairement aux arbres :

- il n'y a pas de racine, donc on doit choisir à partir de quel noeud on part : le **noeud source**.
- il peut y avoir un nombre quelconque d'arêtes, et il faut donc marquer les chemins déjà empruntés lors du parcours.

## 2 Applications

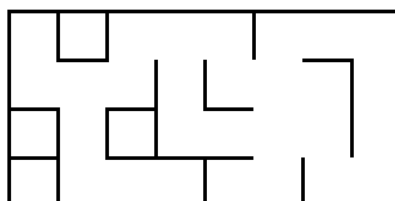
Selon les actions opérées au cours d'un parcours, on peut détecter :

- des cycles dans le graphe,
- trouver le chemin le plus court entre deux sommets,
- calculer la distance entre deux sommets, ..., etc.



Les algorithmes sur les graphes sont très utilisés dans la vie courante, ils permettent par exemple :

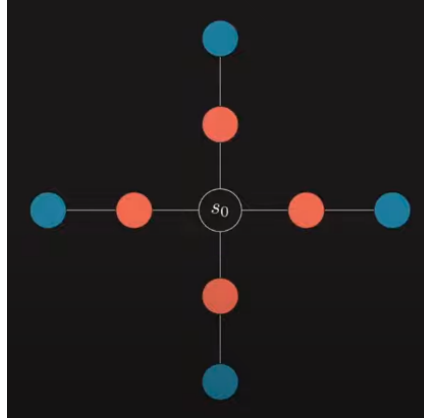
- le routage des paquets de données dans un réseau ;
- de trouver le chemin le plus court entre deux villes (utilisé par les GPS) ;
- de sortir d'un labyrinthe, ..., etc.



### 3 Parcours en largeur

#### 3.1 Principe du *BFS* ou *Breadth-First Search*

A partir d'un sommet  $S_0$ , on explore tous ses voisins (ou successeurs), puis on explore tous les voisins de ces voisins, et ainsi de suite. Le parcours balaie ainsi chaque "branche" au même rythme, d'où le nom de parcours en largeur.



#### 3.2 Algorithme

##### A retenir !

1. On choisit un sommet de départ
2. On l'enfile :
3. Tant que la file n'est pas vide :
  - On défile son premier élément
  - S'il n'a pas encore été visité on le marque et on enfile tous ses voisins non encore visités
  - Sinon, on ne fait rien (on passe donc directement à l'itération suivante)

Regarder cette [vidéo](#) sur le parcours en largeur jusqu'à 7'03.

**Exercice 1 :** On effectue un **parcours en largeur** sur les graphe ci-dessous :

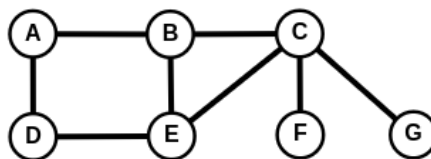


FIGURE 1 – Graphe 1

1. Donner l'ordre dans lequel les sommets sont explorés dans le graphe 1 à partir du sommet A.

2. Même question pour le graphe 2 ci-dessous.

3. Vérifier vos réponses pour **G1** et **G2** avec le site *Graphonline* en allant dans la rubrique *Algorithmes* puis *Parcours en largeur*.

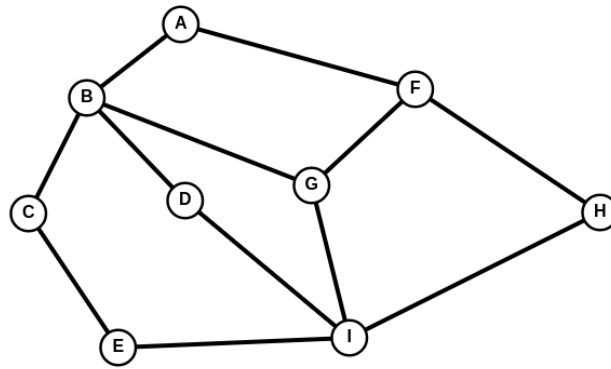


FIGURE 2 – Graphe 2

## 4 Parcours en profondeur

### 4.1 Principe du *DFS* ou *Depth-First Search*

A partir d'un sommet  $S_0$ , on explore un de ses voisins (ou successeurs) et ainsi de suite. S'il n'y a plus de voisins, on revient au sommet précédent et on passe à un autre de ses enfants. Cette façon de faire implique que chaque "branche" est explorée jusqu'au bout, avant de revenir sur nos pas, d'où le nom de parcours en profondeur.



### 4.2 Algorithme

En stockant les sommets encore à visiter dans une pile, on s'assure que ce sont les derniers sommets découverts qui vont être visités en premier (LIFO, *Last In First Out*).

#### A retenir !

1. On choisit un sommet de départ
2. On l'empile
3. Tant que la pile n'est pas vide :
  - On dépile son sommet
  - S'il n'a pas encore été visité on le marque
  - on étudie ses voisins :
    - si le voisin a déjà été visité, on l'ignore
    - si le voisin n'a pas encore été visité, on l'empile et on reprend à l'étape 2
  - Sinon, on ne fait rien (on passe donc directement à l'itération suivante)

Regarder cette [vidéo](#) sur le parcours en profondeur jusqu'à 10'01.

**Exercice 2 :** On effectue un **parcours en profondeur** sur les graphe ci-dessous :

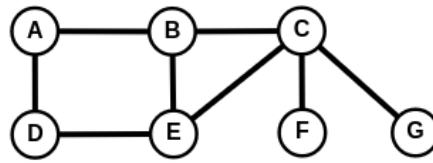


FIGURE 3 – Graphe 1

1. Donner l'ordre dans lequel les sommets sont explorés dans le graphe 1 à partir du sommet A.

2. Même question pour le graphe 2 ci-dessus.

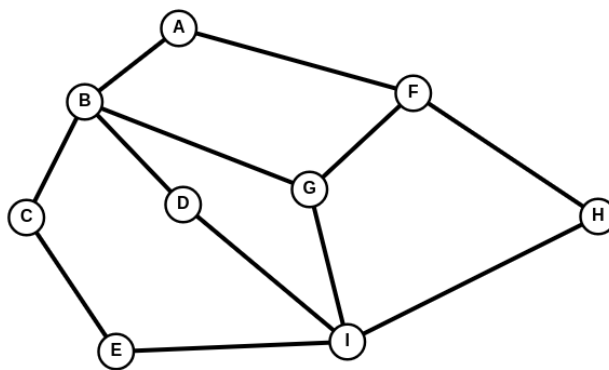


FIGURE 4 – Graphe 2

3. Même question pour le graphe 3 ci-dessous.

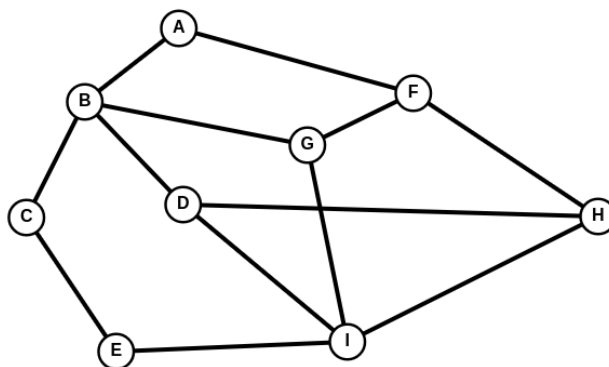


FIGURE 5 – Graphe 3

4. Vérifier vos réponses pour **G1**, **G2** et **G3** avec le site *Graphonline* en allant dans la rubrique *Algorithmes* puis *Parcours en profondeur*.

## 5 Exercices

Dans les exercices ci-dessous, nous utiliserons pour représenter les graphes étudiés par leur liste d'adjacence associée.

### Exercice 3 : Parcours BFS

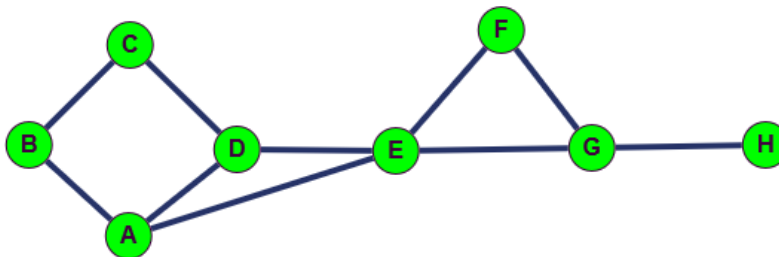
Ecrire le code d'une fonction `parcours_en_largeur` qui parcourt en largeur un graphe à partir du sommet `depart` et dont sa liste d'adjacence est représentée par un dictionnaire nommé `graphe`.

Cette fonction renverra la liste des sommets parcourus en partant du sommet `depart`.

Compléter le script ci-dessous :

```
1 def parcours_en_largeur(graphe, depart):
2     file = [depart]
3     parcours = ...
4     ...
5
6     return parcours
7
8 # Test
9 assert parcours_en_largeur({"A": ["B", "D", "E"], "B": ["A", "C"], "C":
    ["B", "D"], "D": ["A", "C", "E"], "E": ["A", "D", "F", "G"], "F": ["E",
    "G"], "G": ["E", "F", "H"], "H": ["G"]}, "A") == ['A', 'B', 'D', 'E',
    'C', 'F', 'G', 'H']
```

Graphe du test :



### Exercice 4 : BFS optimisé

Le coût d'une recherche dans une liste (pour tester si un sommet a déjà été visité) est beaucoup plus important que le coût de la recherche dans un ensemble (hors programme NSI) ou dans un dictionnaire. (tables de hash)

Nous allons donc proposer une autre version pour améliorer l'efficacité de l'algorithme.

- Un sommet est "vu" lorsqu'il a été mis dans la liste `parcours`
- Un sommet est "en attente" lorsqu'il est dans la file.
- Tous les autres sommets sont "pas vu".

Compléter le script suivant :

```
1 def parcours_largeur_opt(graphe, depart):
2     file = [depart]
3     parcours = []
4     etat = {sommet: "pas vu" for sommet in graphe}
5     etat[depart] = "en attente"
6
7     while file != []:
8         sommet = file.pop(0)
9         parcours.append(sommet)
10        etat[sommet] = ...
```

```

11     for voisin in graphe[sommet]:
12         if etat[voisin] == ...:
13             ...
14     return parcours
15
16 # Test
17 assert parcours_largeur_opt({"A": ["B", "D", "E"], "B": ["A", "C"], "C":
    ["B", "D"], "D": ["A", "C", "E"], "E": ["A", "D", "F", "G"], "F": ["E",
    "G"], "G": ["E", "F", "H"], "H": ["G"]}, "A") == ['A', 'B', 'D', 'E',
    'C', 'F', 'G', 'H']

```

### Exercice 5 : Parcours DFS récursif

Le parcours en profondeur s'écrit naturellement de manière **récursif** :

- L'ensemble des sommets déjà visités est stocké dans un objet Python (liste, ou dictionnaire).
- La visite est relancée récursivement pour chaque voisin du sommet visité qui n'a pas déjà été visité.

Compléter le code de la fonction **récursive** `parcours_profondeur` qui parcourt en profondeur un graphe à partir du sommet `sommet` et dont sa liste d'adjacence est représentée par un dictionnaire nommé `graphe`.

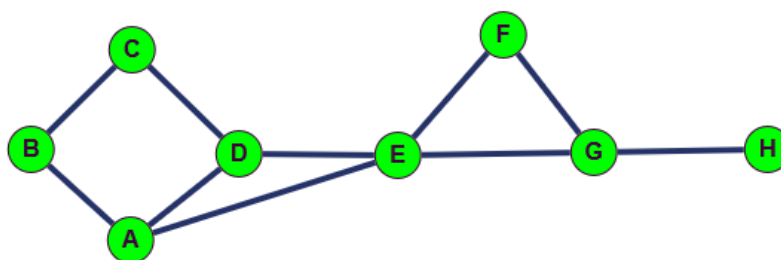
Cette fonction renverra la liste des sommets parcourus en partant du sommet `depart`.

```

1 def parcours_profondeur(graphe, depart, vus = None):
2     if vus is None:
3         vus = []
4         vus.append(depart)
5         for voisin in graphe[depart]:
6             ...
7         return vus
8
9 # Tests
10
11 assert parcours_profondeur({"A": ["B", "D", "E"], "B": ["A", "C"], "C":
    ["B", "D"], "D": ["A", "C", "E"], "E": ["A", "D", "F", "G"], "F": ["E",
    "G"], "G": ["E", "F", "H"], "H": ["G"]}, "A") == ['A', 'B', 'C', 'D',
    'E', 'F', 'G', 'H']

```

Graphe du test :



### Exercice 6 : DFS récursif optimisé

Le coût d'une recherche dans une liste (pour tester si un sommet a déjà été visité) est beaucoup plus important que le coût de la recherche dans un ensemble (hors programme NSI), ou dans un dictionnaire.

Nous allons donc proposer une autre version pour améliorer l'efficacité de l'algorithme.

Compléter la version **récursive** optimisée de la fonction `profondeur_opt`, qui utilise un dictionnaire `vu` pour éviter les doublons :

```

1 def profondeur_opt(graphe, depart, vu = None, parcours = None):
2
3     if vu is None:
4         vu = {sommet : False for sommet in graphe}
5     if parcours is None:
6         parcours = []
7     vu[depart] = True
8     parcours.append(...)
9
10    for voisin in graphe[depart]:
11        if ...:
12            ...
13    return vu, parcours
14
15 # Tests
16 assert profondeur_opt({"A": ["B", "D", "E"], "B": ["A", "C"], "C": ["B",
    "D"], "D": ["A", "C", "E"], "E": ["A", "D", "F", "G"], "F": ["E", "G"],
    "G": ["E", "F", "H"], "H": ["G"]}, "A") == ({'A': True, 'B': True, 'C':
    True, 'D': True, 'E': True, 'F': True, 'G': True, 'H': True}, ['A', 'B',
    'C', 'D', 'E', 'F', 'G', 'H'])

```

### Exercice 7 : DFS itératif avec pile

Comme nous l'avons vu dans la vidéo du cours, on peut aussi utiliser une **pile**.

Ecrire le code d'une fonction `parcours_en_profondeur` qui parcourt en profondeur un graphe à partir du sommet `depart` et dont sa liste d'adjacence est représentée par un dictionnaire nommé `graphe`

Cette fonction renverra la liste des sommets parcourus en partant du sommet `depart`.

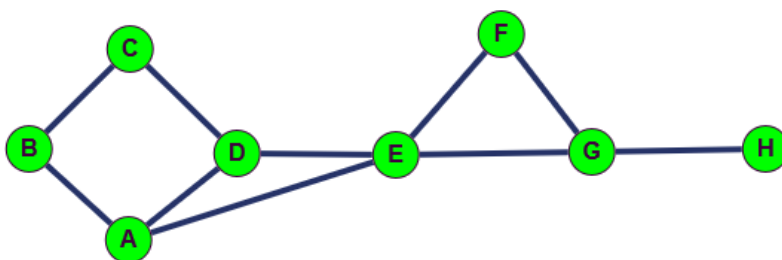
Compléter le script ci-dessous de façon **itérative** :

```

1 def parcours_en_profondeur(graphe, depart):
2     pile = ...
3     parcours = []
4     ...
5     return parcours
6
7 # Tests
8 assert parcours_en_profondeur({"A": ["B", "D", "E"], "B": ["A", "C"], "C":
    ["B", "D"], "D": ["A", "C", "E"], "E": ["A", "D", "F", "G"], "F": ["E",
    "G"], "G": ["E", "F", "H"], "H": ["G"]}, "A") == ['A', 'E', 'G', 'H',
    'F', 'D', 'C', 'B']

```

Graphe du test :



**Exercice 8 : DFS itératif optimisé**

Le coût d'une recherche dans une liste (pour tester si un sommet a déjà été visité) est beaucoup plus important que le coût de la recherche dans un ensemble (hors programme NSI), ou dans un dictionnaire.

Nous allons donc proposer une autre version pour améliorer l'efficacité de l'algorithme :

- Un sommet est "parcouru" lorsqu'il a été mis dans la liste parcours
- Un sommet est "en attente" lorsqu'il est dans la pile.
- Tous les autres sommets sont "pas vu".

Compléter la version **itérative** optimisée de la fonction `profondeur_opt`, qui utilise un dictionnaire `vu` pour éviter les doublons :

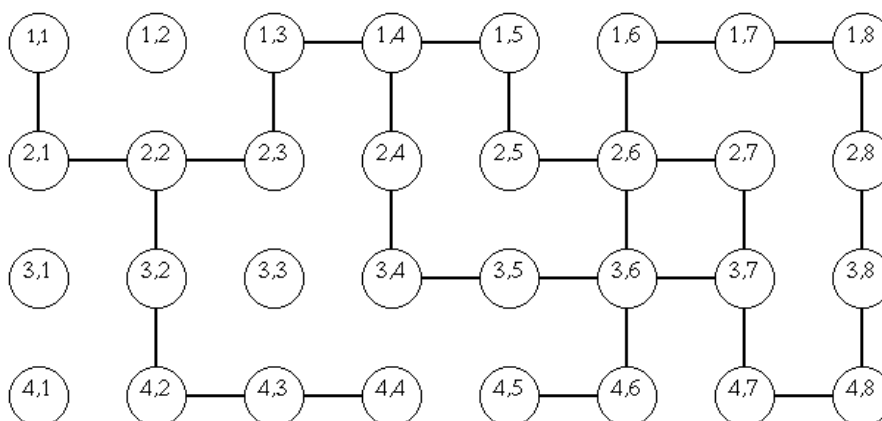
```

1  def parcours_profondeur_iteratif(graphe, depart):
2      parcours = []
3      etat = {sommet: "pas vu" for sommet in graphe}
4      pile = [...]
5      etat[depart] = "en attente"
6      while len(pile) != 0: # ou pile != []
7          sommet = ...
8          parcours.append(...)
9          etat[sommet] = "parcouru"
10         voisins = ...
11         for voisin in ...:
12             if etat[voisin] == "pas vu":
13                 pile.append(...)
14                 etat[voisin] = ...
15     return parcours
16
17 # Tests
18 assert parcours_profondeur_iteratif({"A": ["B", "D", "E"], "B": ["A", "C"],
    "C": ["B", "D"], "D": ["A", "C", "E"], "E": ["A", "D", "F", "G"], "F":
    ["E", "G"], "G": ["E", "F", "H"], "H": ["G"]}, "A") == ['A', 'E',
    'G', 'H', 'F', 'D', 'C', 'B']

```

**Exercice 9 : Labyrinthe**

Suivre les instructions de la [page suivante](#) en écrivant les fonctions manquantes pour résoudre le problème de parcours dans un labyrinthe.





**Exercice 10 : Recherche de chemins dans un graphe**

Sur la rive d'un fleuve se trouvent un loup, une chèvre, un chou et un passeur. Le problème consiste à tous les faire passer sur l'autre rive à l'aide d'une barque, menée par le passeur, en respectant les règles suivantes :

- la chèvre et le chou ne peuvent pas rester sur la même rive sans le passeur ;
- la chèvre et le loup ne peuvent pas rester sur la même rive sans le passeur ;
- le passeur ne peut mettre qu'un seul "passager" avec lui.



On décide de représenter le passeur par la lettre P, la chèvre par la lettre C, le loup par L et le chou par X.

1. Représenter ce problème à l'aide d'un graphe où les sommets sont tous les états possibles sur la rive de départ (par exemple, PLCX est un sommet représentant le fait que tous sont sur la rive de départ.)
2. Trouver une solution au problème en indiquant chacun des déplacements (si possible une solution avec le moins de déplacements possibles).
3. Proposer une modélisation avec un graphe pour résoudre ce jeu.

**Exercice 11 : Annales de Bac**

Afin de s'entraîner pour l'épreuve écrite du Bac, faire les exercices suivants :

- Exercice 2, Amérique du Nord 2024, J1
- Exercice 1, Centre étrangers 2024, J1
- Exercice 2, Asie 2024, J1
- Exercice 2, Métropole 2024, J1
- Exercice 1, Polynésie 2024, J1
- Exercice 1, Polynésie 2024, J2